

SQLOps Guardian

A SQL anti-pattern detection and optimization tool

Linters · RAG · LLM analysis · Operations

Architecture Overview

THE PROBLEM

Bad SQL is silent until it isn't

IN PRODUCTION

Queries that work fine on 10k rows time out on 10M

DELETE without WHERE wipes a table by accident

SELECT * leaks columns and inflates payloads

LIKE '%foo%' kills index usage

The question

Can a single tool flag SQL anti-patterns reliably, explain them, and ground its advice in past examples — without becoming yet another AI black box?

That's the design problem.

A four-layer analysis pipeline

1

Deterministic Linter

10 regex-based anti-pattern rules,
severity-tagged

2

RAG with ChromaDB

Vector search for similar past
cases
(16 seed cases, growable)

3

LLM Analysis

Groq (Llama 3.3 70B) generates
grounded optimization advice

4

Operations

SQLite logging, feedback,
metrics, caching

Orchestrated by [pipeline.py](#) with graceful degradation — each layer fails independently.

What happens on every /analyze call

- 1 Browser → Nginx**
React frontend posts SQL; Nginx reverse-proxies to FastAPI
- 2 FastAPI validates & routes**
Pydantic input validation, then hand off to pipeline orchestrator
- 3 Linter runs**
Regex rules produce findings (~1–5 ms)
- 4 RAG retrieves (ChromaDB)**
Top-K similar past cases by vector distance (~10–50 ms)
- 5 LLM synthesizes (Groq)**
Llama 3.3 70B grounded by linter + RAG context (~800–1500 ms)
- 6 Log & respond (SQLite)**
Persist analysis, return JSON to the client (~1–5 ms)

TOTAL LATENCY

~1–2 s

90% of it is the LLM call.

Everything else combined is ~50 ms. This is what makes caching the highest-leverage optimization.

Two databases, two different jobs

SQLITE

Relational — what happened

Stores: analysis logs, user feedback, metrics, cache

Good for:
exact lookups, counts, aggregations, sorting, updates by ID

Pattern:
write-heavy, append-only, microsecond reads

Single file on disk. Zero ops. WAL mode for concurrency.

CHROMADB

Vector — what we know

Stores: anti-pattern cases as embedded descriptions

Good for:
semantic similarity — "find cases like this query"

Pattern:
read-heavy, curated; grows via accepted feedback

Embedded on disk. No separate service. Cosine similarity.

Embed descriptions, not raw SQL

NAIVE — embed the SQL

```
SELECT * FROM users  
WHERE name LIKE '%smith%'
```

The embedding model sees SQL keywords and table names. Two queries with the same anti-patterns but different tables embed far apart. Two queries with different anti-patterns but shared keywords embed close together.

Surface tokens drown out signal.

CHOSEN — embed a description

Query on users.
Problems: select_star, leading_wildcard_like.
Fix: ...

Both sides — stored cases and incoming queries — are turned into the same descriptive sentence shape before embedding. Similarity now correlates with the thing we actually care about: shared anti-patterns.

Document engineering, not prompt magic.

Boring technology, deliberately chosen

BACKEND

Python 3.12

core runtime

FastAPI

HTTP, validation, async-friendly

Pydantic

request/response models

uv

dependency management

DATA & AI

SQLite

logs, feedback, cache

ChromaDB

embedded vector store

Groq + Llama 3.3 70B

LLM inference

FRONTEND & INFRA

React + Vite + Tailwind

analyzer + dashboard

Recharts

metrics visuals

Docker + Compose

multi-container

Nginx

reverse proxy / static

Four choices worth defending

1

Groq, not Gemini

Gemini free tier: 5 req/min, 20 req/day — unusable. Groq: 30/min, 14,400/day. Latency dropped from ~11 s to ~1.1 s on the same prompt.

2

No LangChain

I needed one ChromaDB call and one Groq call. A framework for that adds a large dependency and an abstraction layer with no benefit at this scale.

3

def, not async def

Every downstream call (Groq, Chroma, SQLite) is synchronous. Sync handlers run on FastAPI's thread pool — true parallelism. Async handlers would block the loop.

4

Graceful degradation

If the LLM fails, the linter and RAG results still return. The pipeline never fails wholesale on a single layer's outage.

What I deliberately didn't build

- ⓧ **API versioning (/v1/)** — There is one version.
- ⓧ **Microservices / Kubernetes** — Single instance handles 6–7 users.
- ⓧ **Redis cache** — SQLite gives microsecond reads at this scale.
- ⓧ **Message queues** — Pipeline responds in 1–2 s. No async work.
- ⓧ **Semantic caching** — Wrong fuzziness model for SQL advice — would return wrong answers.
- ⓧ **JWT authentication** — Public demo. Rate limiting solves the real threat (cost).
- ⓧ **Architecture Decision Records** — Solo project. Decisions live in this deck and the README.

PHILOSOPHY

Every complexity addition has to solve a real problem, not an imagined one.

YAGNI + boring technology. The list above is the project's design, not its limitations.

Where it stands, where it's going

DONE

- Full 4-layer pipeline working end-to-end
- FastAPI backend + React frontend
- Multi-container Docker setup with Nginx
- 107 backend tests passing
- 16 seed cases in ChromaDB
- Migrated Gemini → Groq (10× latency drop)

NEXT

- LLM response caching (SHA-256 keyed in SQLite)
- WAL mode for concurrent SQLite writes
- Rate limiting on /analyze (slowapi, per-IP)
- Auto-seed ChromaDB on first boot
- Deploy to AWS EC2 (single t3.micro, free tier)
- Linter & RAG evaluation harness